

LAB Assignment: Implementing and Comparing Bahdanau vs Luong Attention Mechanisms

Course Name and instructor: GAIL, Prof G C Nandi

Date of announcement: 08-04-2026

Date of submission: 20-04-2026 by mid night (12:00 am)

Total Marks : 50

Learning Objectives

By completing this assignment, students will be able to:

1. **Implement** both additive (Bahdanau) and multiplicative (Luong) attention mechanisms from scratch
2. **Compare** their computational efficiency and memory usage
3. **Analyze** performance differences on sequence-to-sequence tasks
4. **Visualize** attention patterns to understand what each mechanism "sees"
5. **Critique** why Transformer architectures moved to multiplicative attention

What you will do?

You will build a **Neural Machine Translation** system from English to a simplified toy language (or use a real small dataset). You will implement three models:

1. **Baseline**: LSTM Seq2Seq without attention
2. **Model A**: LSTM Seq2Seq with Bahdanau attention
3. **Model B**: LSTM Seq2Seq with Luong attention (dot product variant)

Then, you will compare them across multiple dimensions and write a research report.

Dataset Choose a Toy Dataset (Recommended for speed)

English → "Emoji" translation:

Input: "I am happy" → Output: "🤖 🤖"

Input: "I am sad" → Output: "😞😞"
Input: "very good" → Output: "👍👍"
Input: "I love you" → Output: "❤️🤖🤖"
Input: "machine learning" → Output: "📖📖"

Create 200-500 examples with simple grammar

Implementation Requirements

Part 1: Data Preparation

```
class DataProcessor:
    def __init__(self, source_lang, target_lang, max_len=20):
        """
        Load, tokenize, and create vocabulary
        """
        pass

    def create_embeddings(self, embedding_dim=256):
        """
        Initialize embedding matrices
        """
        pass

    def batchify(self, batch_size=64):
        """
        Create padded batches with masking
        """
```

Deliverables:

- Vocabulary size statistics
- Sequence length distribution plot
- Sample of 10 input-output pairs with tokenization

Part 2: Attention Mechanisms

2.1 Bahdanau (Additive) Attention

```
class BahdanauAttention(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        """
        Implement: score(h_t, h_s) = v^T tanh(W1 h_t + W2 h_s)
        """
        self.W1 = nn.Linear(hidden_dim, hidden_dim, bias=False)
        self.W2 = nn.Linear(hidden_dim, hidden_dim, bias=False)
        self.v = nn.Linear(hidden_dim, 1, bias=False)

    def forward(self, decoder_hidden, encoder_outputs, mask=None):
        """
        Args:
            decoder_hidden: (batch, hidden_dim) - current decoder state
            encoder_outputs: (batch, src_len, hidden_dim) - all encoder states
            mask: (batch, src_len) - padding mask

        Returns:
            context: (batch, hidden_dim) - weighted sum of encoder outputs
            attention_weights: (batch, src_len) - normalized attention scores
        """
```

Key implementation details:

- Use `torch.tanh` for nonlinearity
- Handle variable length sequences with masking
- Apply softmax over the sequence dimension

2.2 Luong (Multiplicative) Attention

```
class LuongAttention(nn.Module):
    def __init__(self, hidden_dim, score_type='dot'):
        super().__init__()
        """
        Implement dot product: score(h_t, h_s) = h_t^T h_s
        Or general: score(h_t, h_s) = h_t^T W h_s
        """
```

```

"""
self.score_type = score_type

if score_type == 'general':
    self.W = nn.Linear(hidden_dim, hidden_dim, bias=False)
elif score_type == 'concat':
    self.W = nn.Linear(2*hidden_dim, hidden_dim)
    self.v = nn.Linear(hidden_dim, 1)

def forward(self, decoder_hidden, encoder_outputs, mask=None):
    """
    Same interface as Bahdanau
    """

```

Implement two Luong variants:

1. dot: Pure dot product
2. general: Learnable weight matrix $h_t^T W h_s$

Part 3: Seq2Seq Model with Attention

```

class AttentionSeq2Seq(nn.Module):
    def __init__(self, vocab_src, vocab_tgt, embedding_dim, hidden_dim,
                  attention_type='bahdanau', num_layers=2, dropout=0.3):
        super().__init__()

        self.encoder = nn.LSTM(embedding_dim, hidden_dim, num_layers,
                                batch_first=True, bidirectional=True, dropout=dropout)

        self.decoder = nn.LSTM(embedding_dim + hidden_dim*2, hidden_dim,
                                num_layers, batch_first=True, dropout=dropout)

        # Choose attention mechanism
        if attention_type == 'bahdanau':
            self.attention = BahdanauAttention(hidden_dim*2)
        elif attention_type == 'luong':
            self.attention = LuongAttention(hidden_dim*2, score_type='dot')
        elif attention_type == 'luong_general':
            self.attention = LuongAttention(hidden_dim*2, score_type='general')

```

```

self.output_projection = nn.Linear(hidden_dim, vocab_tgt)

def forward(self, src, tgt, teacher_forcing_ratio=0.5):
    """
    Training forward pass with teacher forcing
    """
    # YOUR CODE HERE
    pass

def translate(self, src, max_len=50):
    """
    Inference: generate translation without teacher forcing
    """

```

Critical requirements:

- Bidirectional encoder (states concatenated)
- Attention context vector concatenated with decoder input at each step
- Proper handling of padding and masking
- Gradient clipping to avoid explosion

Part 4: Training and Evaluation

```

def train_model(model, train_loader, val_loader, epochs=20, lr=0.001):
    """
    Training loop with:
    - Teacher forcing ratio scheduling
    - Checkpoint saving
    - Early stopping
    """

def evaluate_model(model, test_loader):
    """
    Metrics to compute:
    - BLEU score
    - Exact match accuracy
    - Perplexity
    """

```

- Inference time per sentence

Comparative Analysis Tasks

Task A: Performance Metrics Comparison

Train all three models (Baseline, Bahdanau, Luong) and compare:

Metric	Baseline	Bahdanau	Luong (dot)	Luong (general)
Final BLEU Score				
Training Time (hours)				
Inference Time (ms/sent)				
Memory Usage (GB)				
# Parameters				

Plot these:

- 1. Training loss curves (all models on same axes)
- 2. Validation BLEU over epochs
- 3. Attention heatmaps for sample sentences

Task B: Computational Complexity Analysis

Theoretical analysis:

Given:

- Source length = S
- Target length = T
- Hidden dimension = H
- Batch size = B

Compute FLOPs for attention computation per decoder step:

Bahdanau: $O(S \times H^2)$ (due to neural network per position)

Luong (dot): $O(S \times H)$ (just dot product)

Luong (general): $O(S \times H^2)$ but with matrix multiplication

Time your actual implementations to verify.

Report:

- Wall-clock time for 100 forward passes with $S=10,50,100,200$
- Plot time vs sequence length for both attention types
- Explain why Luong scales better

Task C: Qualitative Analysis

Pick 10 test sentences and visualize attention weights:

```
def visualize_attention(model, src_sentence, tgt_sentence=None):
    """
    Generate attention heatmap for each decoder step
    """
```

Questions to answer:

1. Does Bahdanau produce "sharper" attention distributions?
2. Does Luong sometimes attend to multiple positions equally?
3. Provide examples where one attention mechanism fails but other succeeds

Extension Challenges (Extra Credit)

Challenge 1: Implementation from Scratch (NumPy only)

Implement both attention mechanisms without PyTorch's autograd. Use only NumPy and implement manual backpropagation.

Challenge 2: Memory Efficiency

Implement **sparse attention** that only attends to k nearest positions (reduces $O(S^2)$ to $O(S \times k)$)

Challenge 3: Visualization Dashboard

Create an interactive Streamlit/Gradio app showing:

- Real-time attention visualization
- Model comparison side-by-side
- Attention interpolation between layers

Challenge 4: Transformer Baseline

Implement a small Transformer and compare against both attention mechanisms:

- Transformer vs LSTM+Bahdanau vs LSTM+Luong
- Discuss why Transformer wins despite using Luong-style attention

You may take help from any foundational models to learn this assignemnet.